Понятия UpCast и DownCast. Понятие полиморфизма. Использование герметизированных классов



IThub

↑ UpCast (восходящее приведение)

- Дочерний класс → Родительский класс

Upcast происходит, когда объект с более конкретным типом приводится к типу, который более общий

```
class Animal {
    void sound() {
        System.out.println("Звук животного");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Мяу!");
    }
}
```

```
Cat cat = new Cat();

Animal animal = cat; // UpCast
```

Теперь объект Cat рассматривается как Animal.

✅ Безопасно
✅ Выполняется автоматически
❌ Доступны только методы родительского класса

IThub

↓ DownCast (нисходящее приведение)

- Родительский класс → Дочерний класс

Downcast происходит, когда объект с более общим типом приводится к типу, который более конкретный

```
class Animal {
    void sound() {
        System.out.println("Звук животного");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Мяу!");
    }
}
```

```
Animal animal = new Cat();
Cat cat = (Cat) animal; // DownCast
```

✅ Теперь можно использовать методы класса Cat

IThub

IThub

generic classes

- **Герметизированные классы, обобщения** или **дженерики** (generic classes) в Java – это классы, которые могут работать с различными типами данных.
- с использованием обобщений можно создать единственный класс, который автоматически работает с разными типами данных
- класс, интерфейс или метод, который оперирует на параметризованном типе, называется **обобщенным**
- обобщения работают только со ссылочными типами (примитивные типы вроде int или char применять нельзя)

```java
public class Gen<T> {

    private T value;

    public Gen(T value) {
        this.value = value;
    }

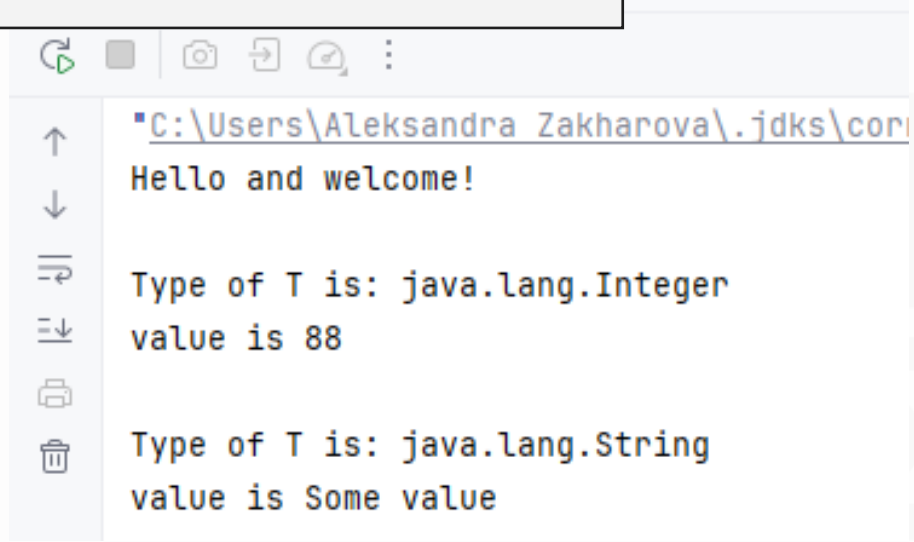
    public T getValue() {
        return value;
    }

    void showType() {
        System.out.println("Type of T is: " + value.getClass().getName());
    }
}
```

```java
    // для объектов типа Integer
    Gen<Integer> objGenInt;
    objGenInt = new Gen<Integer>(88);
    objGenInt.showType();
    int value = objGenInt.getValue();
    System.out.println("value is " + value);

    System.out.println();

    // для объекто типа String
    Gen<String> objGenString;
    objGenString = new Gen<String>("Some value");
    objGenString.showType();
    String valueString = objGenString.getValue();
    System.out.println("value is " + valueString);
```



IThub

- Герметизированные классы также позволяют повысить переиспользуемость кода. Мы можем написать один герметизированный класс, который может работать с различными типами данных, вместо того чтобы писать отдельный класс для каждого типа данных. Это упрощает и ускоряет разработку программного обеспечения.
- В Java также есть герметизированные методы (generic methods), которые могут работать с различными типами данных. Они могут быть объявлены в герметизированном или не герметизированном классе.

Задание

IThub

1. Вам **необходимо создать универсальный класс** Box<T>, который будет представлять коробку для хранения одного объекта.

Класс должен содержать:

- Поле value — значение, которое хранится в коробке.
- Тип поля должен быть универсальным: T

2. Создайте методы

- Сохранение значения: **setValue(T value)** - метод записывает объект в коробку.
- Получение значения: **T getValue()** - метод возвращает объект из коробки.
- Вывод информации: **showInfo()** - метод должен выводить: В коробке хранится: ...

3. **Используйте класс с разными типами данных**

Создайте несколько объектов:

- Коробка с числом:

Box<Integer> numberBox

Сохраните:100 (метод **setValue)**

- Коробка со строкой:

Box<String> textBox

Сохраните: "Java "(метод **setValue)**

4. Создайте класс: Book

с полями:

- название
- автор

Создайте: Box<Book> bookBox и положите туда книгу (созданную через конструктор метод **setValue**)

Выведи информацию, что хранится в коробке (метод **showInfo)**